1.Book

Source file: Q1_Book.cpp

Observed output:

```
Terminal Output — Q1_Book

C:\> g++ -std=c++17 -Wall -Wextra -pedantic Q1_Book.cpp -o Q1_Book
C:\> Q1_Book.exe

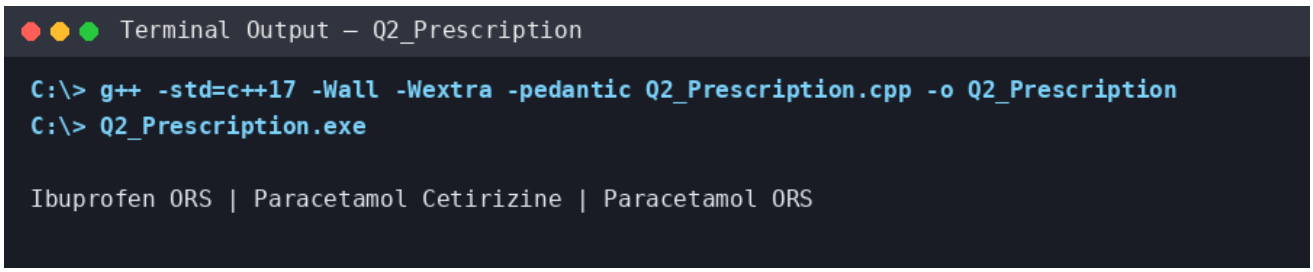
Title: (empty), Author: , Copies: 0
Title: C++ Basics, Author: Unknown, Copies: 1
Title: Clean Code, Author: Robert Martin, Copies: 4
Error: Copies cannot be negative
```

Source code

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;
class Book {
    string title, author;
    int copies;
    static int checkedCopies(int n) {
        if (n < 0) throw invalid_argument("Copies cannot be negative");
        return n;
    }
public:
    Book() : Book("", "", 0) {}
    explicit Book(string t) : Book(t, "Unknown", 1) {}
    Book(string t, string a, int n) : title(t), author(a), copies(checkedCopies(n)) {}
    void print() const {
        cout << "Title: " << (title.empty() ? "(empty)" : title)
             << ", Author: " << author << ", Copies: " << copies << '\n';
    }
};
int main() {
    Book empty; Book donated("C++ Basics"); Book full("Clean Code", "Robert Martin", 4);
    empty.print(); donated.print(); full.print();
    try { Book invalid("Bad record", "Nobody", -2); invalid.print(); }
    catch (const invalid_argument& e) { cout << "Error: " << e.what() << '\n'; }
}
```

2. Prescription
Source file: Q2_Prescription.cpp

Observed output:

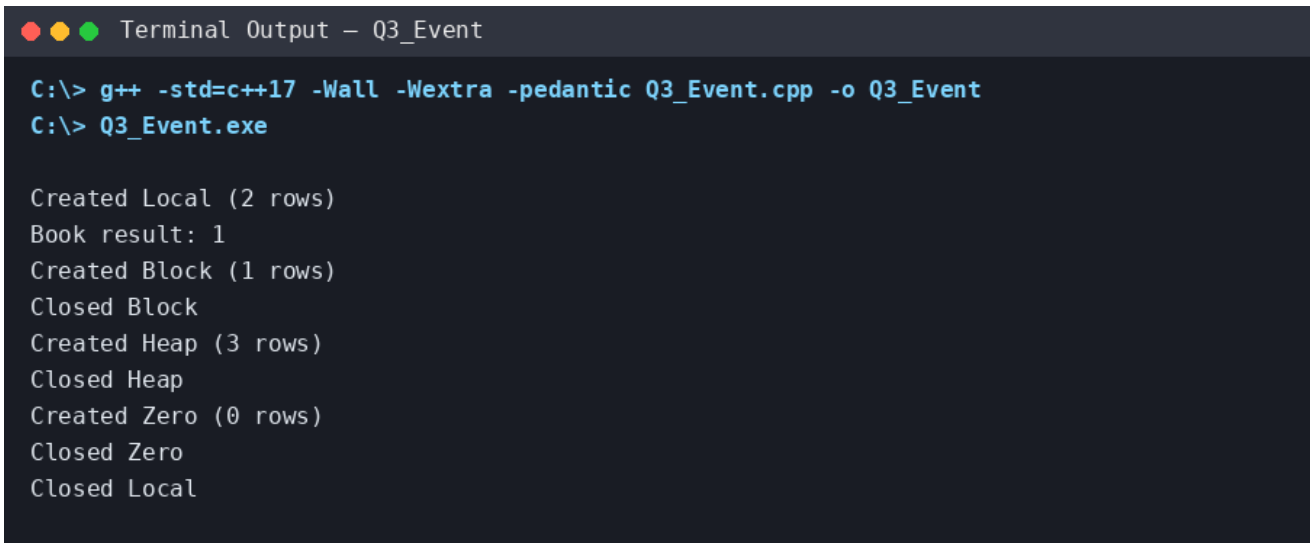


Source code

```
#include <iostream>
#include <string>
class Prescription {
   int n; std::string* medicines;
public:
   explicit Prescription(int count) : n(count), medicines(count > 0 ? new std::string[count] : nullptr) {}
   Prescription(const Prescription& o) : n(o.n), medicines(o.n > 0 ? new std::string[o.n] : nullptr) {
      for (int i=0;i<n;++i) medicines[i]=o.medicines[i];
   }
   Prescription& operator=(const Prescription& o) {
      if (this != &o) {
         std::string* fresh=o.n > 0 ? new std::string[o.n] : nullptr;
         for(int i=0;i<o.n;++i) fresh[i]=o.medicines[i];
         delete[] medicines; medicines=fresh; n=o.n;
      }
      return *this;
   }
   ~Prescription(){ delete[] medicines; }
   void set(int i,const std::string& m){ if(i>=0&&i<n) medicines[i]=m; }
   std::string get(int i)const{return (i>=0&&i<n)?medicines[i]:"";}
};
int main(){
   Prescription p(2); p.set(0,"Paracetamol"); p.set(1,"ORS");
   Prescription q=p; q.set(1,"Cetirizine");
   Prescription r(1); r=p; p.set(0,"Ibuprofen");
   std::cout<<p.get(0)<<' '<<p.get(1)<<" | "<<q.get(0)<<' '<<q.get(1)<<" | "<<r.get(0)<<'
'<<r.get(1)<<'\n';
}
```

3. Event
Source file: Q3_Event.cpp

Observed output:



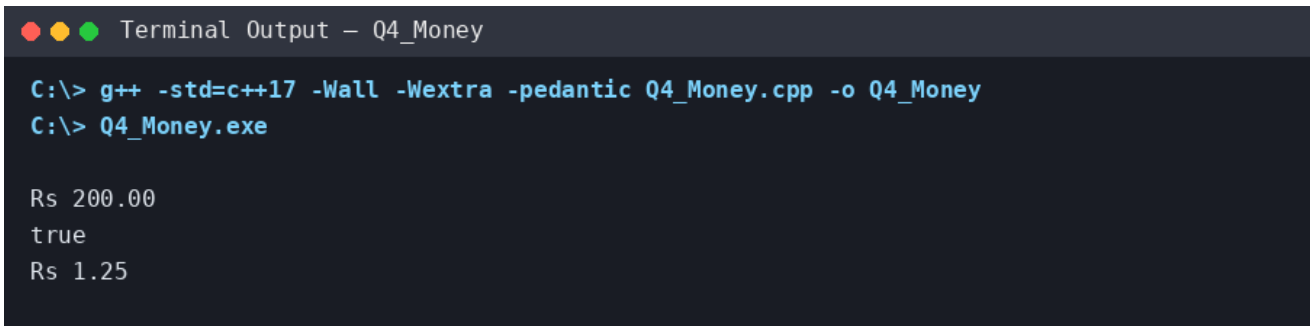
Source code

```
#include <iostream>
#include <string>
#include <stdexcept>
class Event {
   int rows, cols; bool** seats; std::string name;
public:
   Event(std::string n,int r,int c):rows(r),cols(c),seats(nullptr),name(n){
      if(r<0||c<0) throw std::invalid_argument("Negative dimensions");
      if(rows){ seats=new bool*[rows]{}; try { for(int i=0;i<rows;++i) seats[i]=new bool[cols]{}; }
      catch(...){for(int i=0;i<rows;++i) delete[] seats[i]; delete[] seats; throw;} }
      std::cout<<"Created "<<name<<" ("<<rows<<" rows)\n";
   }
   Event(const Event&)=delete; Event& operator=(const Event&)=delete;
   bool book(int r,int c){if(!seats||r<0||r>=rows||c<0||c>=cols||seats[r][c])return false;seats[r][c]=true;return
true;}
   ~Event(){for(int i=0;i<rows;++i)delete[] seats[i];delete[] seats;std::cout<<"Closed "<<name<<'\n';}
};
int main(){
   Event local("Local",2,3); std::cout<<"Book result: "<<local.book(0,1)<<'\n';
   {Event block("Block",1,2);block.book(0,0);}
   Event* heap=new Event("Heap",3,4);heap->book(1,1);delete heap;
   Event zero("Zero",0,5);
}
```

4. Money

Source file: Q4_Money.cpp

Observed output:



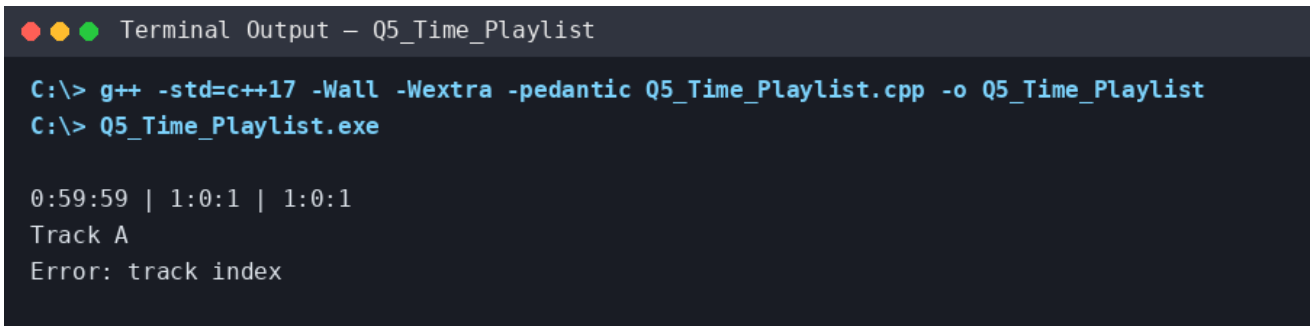
Source code

```
#include <iomanip>
#include <iostream>
class Money {
   long long paise;
public:
   Money(long long r=0,int p=0):paise(r*100+p){}
   Money operator+(const Money& o)const{Money x;x.paise=paise+o.paise;return x;}
   bool operator==(const Money& o)const{return paise==o.paise;}
   friend std::ostream& operator<<(std::ostream& out,const Money& m){
      long long whole=m.paise/100, rem=m.paise%100;
      if(rem<0)rem=-rem;
      if(m.paise<0&&whole==0)out<<"Rs -0."<<std::setw(2)<<std::setfill('0')<<rem;
      else out<<"Rs "<<whole<<'.'<<std::setw(2)<<std::setfill('0')<<rem;
      out<<std::setfill(' ');return out;
   }
};
int main(){
   Money a(120,75),b(79,25);Money c=a+b;
   std::cout<<c<<'\n'<<std::boolalpha<<(c==Money(200))<<'\n';
   std::cout<<Money(0,75)+Money(0,50)<<'\n';
}
```

5. Time Playlist

Source file: Q5_Time_Playlist.cpp

Observed output:

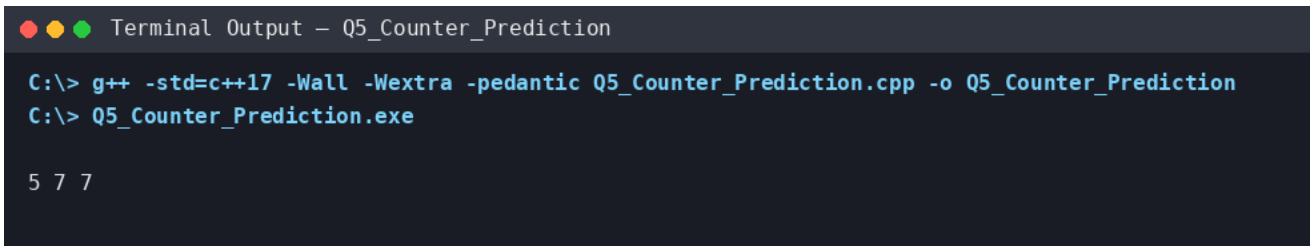


Source code

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
class Time {
   int seconds;
public:
   explicit Time(int s=0):seconds((s%86400+86400)%86400){}
   Time& operator++(){seconds=(seconds+1)%86400;return *this;}
   Time operator++(int){Time old=*this;++(*this);return old;}
   bool operator==(const Time& x)const{return seconds==x.seconds;}
   bool operator<(const Time& x)const{return seconds<x.seconds;}
   friend std::ostream& operator<<(std::ostream& o,const Time& t){
      return o<<t.seconds/3600<<':'<<(t.seconds/60)%60<<':'<<t.seconds%60;
   }
};
class Playlist {
   std::vector<std::string> tracks;
public:
   explicit Playlist(std::vector<std::string> t):tracks(t){}
   std::string& operator[](int i){if(i<0||i>=static_cast<int>(tracks.size()))throw std::out_of_range("track
index");return tracks[i];}
   const std::string& operator[](int i)const{if(i<0||i>=static_cast<int>(tracks.size()))throw
std::out_of_range("track index");return tracks[i];}
};
int main(){
   Time t(59*60+59);Time a=t++;Time b=++t;
   std::cout<<a<<" | "<<b<<" | "<<t<<'\n';
   Playlist p({"Track A","Track B"});std::cout<<p[0]<<'\n';
   try{std::cout<<p[-1];}catch(const std::out_of_range& e){std::cout<<"Error: "<<e.what()<<'\n';}
```

6. Counter Prediction
Source file: Q5_Counter_Prediction.cpp

Observed output:



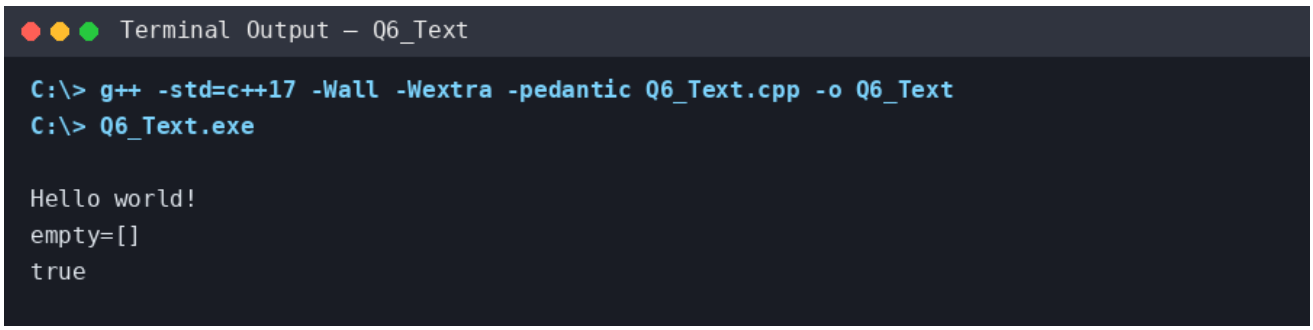
Source code

```
#include <iostream>
struct Counter {
    int v;
    Counter& operator++(){++v;return *this;}
    Counter operator++(int){Counter old=*this;++v;return old;}
};
int main(){Counter c{5};Counter a=c++;Counter b=++c;std::cout<<a.v<<' '<<b.v<<' '<<c.v<<'\n';}
```

Notes
Expected result is 5 7 7: postfix returns the old value; prefix returns the incremented value.

7. Text
Source file: Q6_Text.cpp

Observed output:



Source code

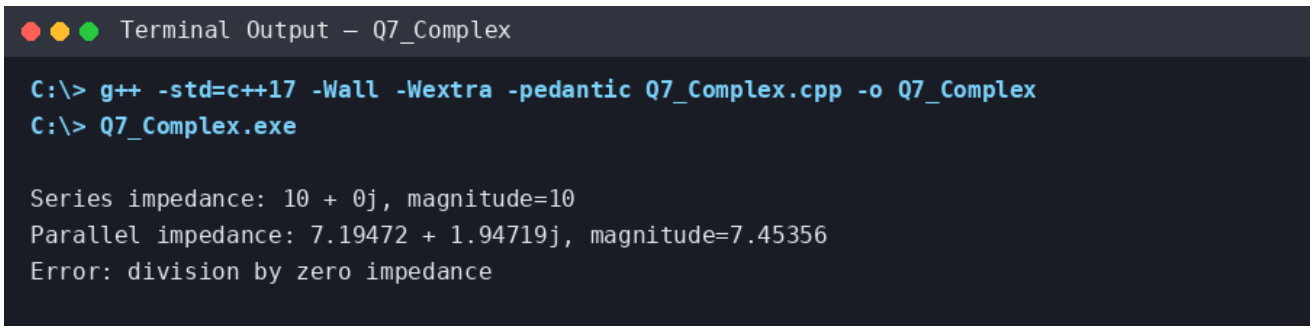
```
#include <cstring>
#include <iostream>
class Text {
    char* data;
public:
    Text(const char* s=""){if(!s)s="";data=new char[std::strlen(s)+1];std::strcpy(data,s);}
    Text(const Text& o):Text(o.data){}
    Text& operator=(const Text& o){if(this!=&o){char* p=new
char[std::strlen(o.data)+1];std::strcpy(p,o.data);delete[] data;data=p;}return *this;}
    ~Text(){delete[] data;}
    Text& operator+=(const Text& rhs){std::size_t a=std::strlen(data),b=std::strlen(rhs.data);char* p=new
char[a+b+1];std::memcpy(p,data,a);std::memcpy(p+a,rhs.data,b+1);delete[] data;data=p;return *this;}
    friend Text operator+(const Text& a,const Text& b){Text r(a);r+=b;return r;}
    bool operator==(const Text& r)const{return std::strcmp(data,r.data)==0;}
    bool operator!=(const Text& r)const{return !(*this==r);}
    friend std::ostream& operator<<(std::ostream& o,const Text& t){return o<<t.data;}
};
int main(){Text a("Hello "),b("world"),c("!");Text result=a+b+c;std::cout<<result<<'\n';Text
empty;std::cout<<"empty=["<<empty<<"]\n";a=a;std::cout<<std::boolalpha<<(a!=b)<<'\n';}
```

Notes
Demonstrates concatenation, empty text and self-assignment.

8. Complex

Source file: Q7_Complex.cpp

Observed output:



Source code

```
#include <cmath>
#include <iostream>
#include <stdexcept>
class Complex {
   double re,im;
public:
   Complex():re(0),im(0){}
   Complex(double r):re(r),im(0){}
   Complex(double r,double i):re(r),im(i){}
   double magnitude()const{return std::sqrt(re*re+im*im);}
   Complex operator+(const Complex& z)const{return {re+z.re,im+z.im};}
   Complex operator-(const Complex& z)const{return {re-z.re,im-z.im};}
   Complex operator*(const Complex& z)const{return {re*z.re-im*z.im,re*z.im+im*z.re};}
   Complex operator/(const Complex& z)const{double d=z.re*z.re+z.im*z.im;if(d==0.0)throw
std::domain_error("division by zero impedance");return {(re*z.re+im*z.im)/d,(im*z.re-re*z.im)/d};}
   bool operator==(const Complex& z)const{return re==z.re&&im==z.im;}
   friend std::ostream& operator<<(std::ostream& o,const Complex& z){return o<<z.re<<(z.im<0?" - ":"
+ ")<<std::abs(z.im)<<"j";}
};
int main(){
   const double R=10,w=100,L=0.1,C=0.001;
   Complex zr(R,0),zl(0,w*L),zc(0,-1.0/(w*C));Complex series=zr+zl+zc;
   std::cout<<"Series impedance: "<<series<<", magnitude="<<series.magnitude()<<'\n';
   Complex z1(10,5),z2(20,-2);Complex parallel=(z1*z2)/(z1+z2);
   std::cout<<"Parallel impedance: "<<parallel<<", magnitude="<<parallel.magnitude()<<'\n';
   try{Complex bad=z1/Complex(0,0);std::cout<<bad<<'\n';}
   catch(const std::domain_error& e){std::cout<<"Error: "<<e.what()<<'\n';}
}
```